

INGENIERÍA EN SISTEMAS DE INFORMACIÓN

Algoritmos y Estructuras de Datos

Comisión: 4

TRABAJO FINAL INTEGRADOR LABORATORIO DE PYTHON

Sistema de Gestión de Hotel

Integrantes:

*Peralta Luciano Iván
Ramírez Pablo Nicolás
Rojas Gonzalo
Vaernet Bianca Estefanía
Zarate Brisa*

Grupo: D9

Profesores:

*Bolatti Diego
Robles Axel
Hodapp Ilse*

~ Ciclo Lectivo 2026 ~

1. Introducción.....	2
2. Estructura del programa.....	2
3. Explicación de las funciones principales.....	3
4.Estructuras de datos definitivas.....	4
5.Estructuras de programación utilizadas.....	5
6. Algoritmos importantes.....	6
7. Casos de uso.....	6
8. Capturas del sistema.....	7
9. Uso de Inteligencia Artificial.....	7
10. Trabajo colaborativo.....	8
11. Conclusión.....	8
12. Anexos.....	8

1. Introducción

Escenario asignado

- **10. Gestión de hotel**

El sistema deberá administrar reservas y ocupación de habitaciones dentro de un hotel. Podrá contemplar registro de huéspedes, check-in, check-out, control de habitaciones disponibles y cálculo de estadías.

La solución también podrá incorporar distintos tipos de habitaciones y estadísticas de ocupación.

Objetivo del proyecto

Este proyecto nace con el objetivo de cumplir el Trabajo Final de Python. Nuestra aplicación es un sistema de consola diseñado para digitalizar la recepción de un hotel, permitiendo administrar el flujo de clientes, el inventario de habitaciones y las cuentas del negocio desde un menú interactivo.

Para que el proyecto se considere exitoso, el sistema apunta a cumplir con los siguientes objetivos principales:

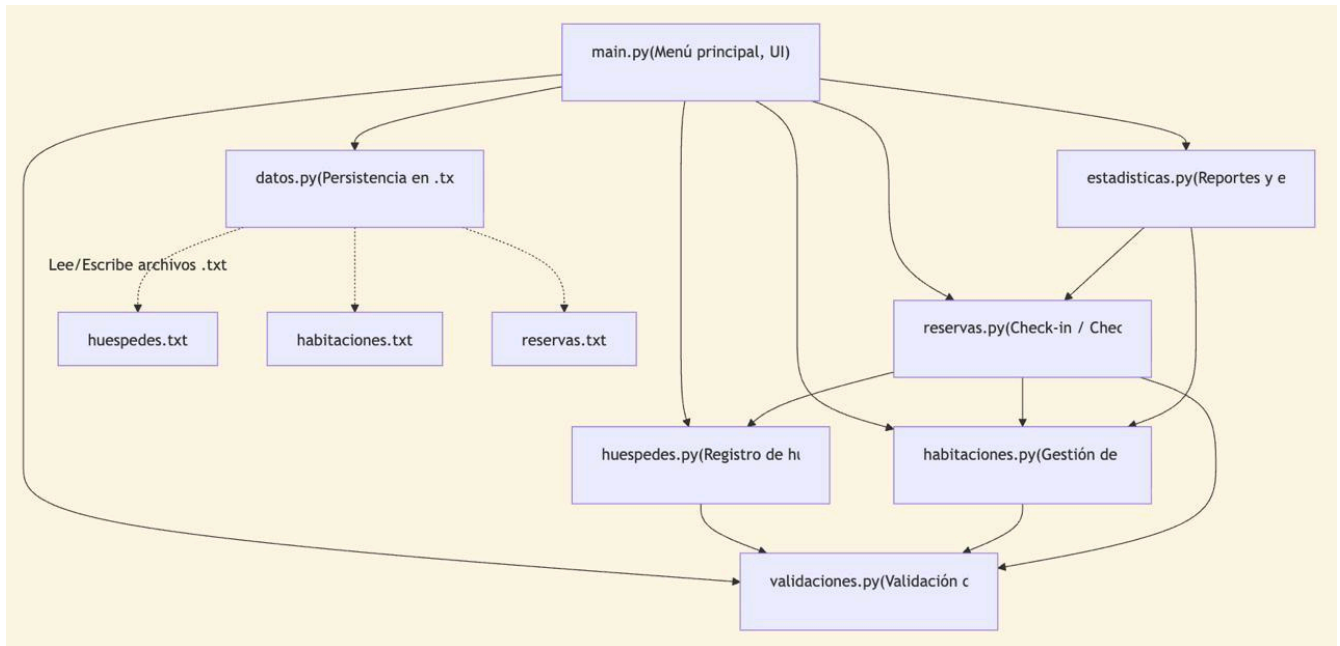
- **Automatizar la gestión hotelera:** Reemplazar el uso de planillas físicas o manuales para centralizar en un solo lugar el control de los huéspedes, la ocupación de los cuartos y la facturación de las estadías.
- **Garantizar la seguridad de los datos:** Asegurar que toda la información con la que se trabaja en el día a día quede resguardada de forma permanente al cerrar la aplicación, evitando pérdidas de datos o cierres inesperados del programa por errores de tipeo.
- **Ofrecer una herramienta ágil y resolutive:** Brindar al usuario una interfaz intuitiva que le permita conocer el estado del hotel en tiempo real, obtener reportes de ingresos inmediatos y cancelar cualquier operación de forma simple si se comete un error.

2. Estructura del programa

gestion_hotel/

— main.py	# Punto de entrada, menú principal, UI (limpiar_pantalla)
— habitaciones.py	# Funciones de gestión de habitaciones
— huespedes.py	# Funciones de gestión de huéspedes
— reservas.py	# Funciones de check-in y check-out
— estadisticas.py	# Funciones de reportes y estadísticas
— validaciones.py	# Funciones de validación (incluyendo escape con '0')
— datos.py	# Funciones de persistencia (archivos .txt con pipe)
— README.md	# Documentación general del proyecto

└─ datos/ # Carpeta generada dinámicamente con la data
 ├─ huespedes.txt # Base de datos de clientes
 ├─ habitaciones.txt # Base de datos de cuartos
 └─ reservas.txt # Base de datos de estadías



3. Explicación de las funciones principales

Para cumplir con el requerimiento de modularización, organizamos el código de la aplicación dividiendo las responsabilidades en distintos archivos Python. Esta arquitectura permite trabajar en paralelo, aislar los errores y evita tener un único código excesivamente largo.

3.1 Punto de entrada (main.py):

Es el archivo principal que se debe ejecutar para iniciar la aplicación. Contiene el bucle principal (while) que mantiene el sistema activo y se encarga exclusivamente de la interfaz de usuario por consola (dibujar los menús en pantalla y redirigir al operador según la opción elegida).

3.2 Módulo de gestión de clientes (huespedes.py)

Concentra la lógica que manipula los datos en la memoria RAM dentro de lista_huespedes. La función registrar_huesped pide el DNI, aplica una **búsqueda secuencial** para evitar duplicados y, si el cliente es nuevo, guarda sus datos en un diccionario con .append(). Para las consultas, ver_huespedes recorre la estructura de punta a punta con un bucle for para mostrar la tabla completa, mientras que buscar_huesped filtra por un DNI específico. Por último, eliminar_huesped borra al cliente por referencia únicamente tras realizar un control cruzado y verificar que no tenga ninguna reserva activa.

3.3 Módulo de gestión de habitaciones (habitaciones.py)

Controla el estado del hotel a través de `lista_habitaciones`. La función `mostrar_habitaciones` detalla en consola el número, tipo, precio y si el cuarto está disponible o ocupado. Como soporte clave, `obtener_disponibles` recorre la lista y devuelve una sublista con los cuartos libres, lo cual es fundamental para el Check-in ya que evita sobre-asignar una habitación en uso.

3.4 Módulo de gestión de reservas (reservas.py)

Es el núcleo transaccional que conecta huéspedes y habitaciones mediante `lista_reservas`. El proceso `hacer_checkin` valida el DNI del cliente, muestra los cuartos libres por categoría y, al confirmar, pasa la habitación a "ocupada", calcula un ID autoincremental (máximo + 1) y crea la reserva "activa". Para el Check-out, `hacer_checkout` localiza la reserva, aplica la **fórmula matemática** ($\text{noches} * \text{precio}$) para facturar y, tras confirmar el pago, cierra la reserva como "finalizada" y vuelve a poner el cuarto "disponible".

3.5 Módulo de estadísticas (estadisticas.py)

Es la capa analítica del hotel que procesa las listas en la RAM. Cuenta con funciones que recorren las estructuras usando bucles for combinados con **contadores** para calcular el porcentaje de ocupación en tiempo real y **acumuladores** para sumar los montos de las reservas terminadas en el total de ingresos. También incluye clasificaciones por tipo de cuarto y un algoritmo de **búsqueda de máximo** para encontrar al huésped con el récord de noches acumuladas.

3.6 Módulo de validaciones generales (validaciones.py)

Funciona como nuestra barrera de seguridad transversal para sanitizar las entradas del teclado. Tiene funciones como `pedir_dni` que fuerzan formatos estrictos (8 caracteres numéricos sin letras). Lo más importante es que implementa bucles while de re-intento que, si el operador ingresa un "0", cortan el ciclo y devuelve None. Esto actúa como un **mecanismo de escape seguro** que le avisa a los demás módulos que deben cancelar la transacción actual sin corromper los datos.

3.7 Módulo de persistencia de datos (datos.py)

Se encarga exclusivamente de la comunicación con el almacenamiento físico en disco. Al arrancar el programa, abre los archivos .txt en modo lectura, procesa las cadenas usando el separador | y levanta la información para construir las listas de diccionarios en la RAM; si el archivo no existe, un bloque try/except atrapa el FileNotFoundError y crea la lista vacía. Al elegir salir, recorre las listas con un for y vuelca todo de regreso a los archivos planos en modo escritura ("w").

4. Estructuras de datos definitivas

Se usaron **listas de diccionarios** para representar los conjuntos de registros (concepto de archivo en teoría).

- **Huéspedes**

Campo clave: "dni"

```
{  
    "dni": "12345678",  
    "nombre": "Juan Pérez",
```

```
"telefono": "1155667788"
}
```

- **Habitaciones**

Campo clave: "numero"

Tipos: "simple", "doble", "suite"

Estados: "disponible", "ocupada", "mantenimiento"

```
{
    "numero": 101,
    "tipo": "simple",
    "precio_noche": 25000.0,
    "estado": "disponible"
}
```

- **Reservas**

Campo clave: "id_reserva"

Estado: "activa", "finalizada"

```
{
    "id_reserva": 1,
    "dni_huesped": "12345678",
    "numero_habitacion": 101,
    "noches": 3,
    "estado": "activa"
}
```

5. Estructuras de programación utilizadas

Concepto de la cátedra	Donde se aplicó
Condicional simple (if)	Verificaciones de cancelación (<code>if dato is None: return False</code>).
Condicional múltiple (if-elif-else)	Derivaciones en los menús de <code>main.py</code> .
Bucle while (iteración indefinida)	Menú principal y bucles de control en <code>validaciones.py</code> .
Bucle for (iteración definida)	Recorrer listados (ej: ver huéspedes).
Contadores	<code>cont += 1</code> en listados de huéspedes o al contar habitaciones ocupadas.
Acumuladores	<code>total_ingresos += monto</code> en estadísticas.
Banderas	<code>encontrado = False</code> , <code>valor_valido = False</code> .
Paso por referencia	Las funciones reciben <code>lista_huespedes</code> y la modifican in-place con <code>.append()</code> .
with open y try/except	Toda la lógica de <code>datos.py</code> al leer los archivos <code>.txt</code> .
Asignación de expresión	<code>monto_total = hab["precio_noche"] * noches</code> .

6. Algoritmos importantes

Búsqueda Secuencial con Interrupción: Se implementa para el control de duplicados y la vinculación de entidades. El algoritmo toma un dato clave de entrada (como un DNI o número de habitación) y recorre de forma lineal el arreglo dinámico mediante un ciclo definido. En cada iteración, compara la clave buscada con el campo correspondiente del diccionario; si la coincidencia es verdadera, interrumpe el bucle inmediatamente (break) para ahorrar recursos en memoria y retorna la estructura encontrada. Si el ciclo finaliza exhaustivamente sin interrupciones, determina la inexistencia del registro.

Cálculo de Extremos (Máximos) y Acumulación: Utilizado para procesar volúmenes de datos históricos. El algoritmo de acumulación inicializa una variable numérica en cero y realiza un recorrido secuencial sumando los valores de forma transaccional. Para determinar el elemento máximo, se utiliza una variable testigo inicializada en el límite inferior (cero); a medida que se itera el ordenamiento de los datos, cualquier valor que supere al testigo lo reemplaza, actualizando también la referencia del registro para identificar al puntero ganador al final del ciclo.

7. Casos de uso

Caso 1: Flujo Normal de Carga Inexistente

- Paso a paso: El operador inicia la opción de registro, ingresa una clave de identificación nueva, completa los campos requeridos por los parámetros del sistema y confirma la acción.
- Resultado: El sistema valida la ausencia de duplicados, inserta el nuevo diccionario en la lista de la memoria RAM e impacta los cambios con un mensaje de éxito en la consola.

Caso 2: Intercepción de Anomalías (Flujo Alternativo)

- Paso a paso: El operador ingresa un tipo de dato erróneo (caracteres alfabéticos en campos numéricos) o una opción fuera de los rangos válidos del menú.
- Resultado: El módulo de control perimetral captura el error impidiendo el colapso del programa, despliega una advertencia en pantalla y mantiene el bucle de reintento activo sin alterar la persistencia del disco.

Caso 3: Procesamiento Analítico

- Paso a paso: El usuario solicita la pantalla de reportes consolidados del negocio.
- Resultado: El sistema computa las variables dinámicas en milisegundos y devuelve en la interfaz los resultados cuantitativos calculados (totales, promedios e indicadores de extremos).

8. Capturas del sistema

SISTEMA DE GESTION HOTELERA

- Gestion de huéspedes
- Gestion de habitaciones
- Reservas (check-in / ver)
- Check-out
- Estadísticas
- Guardar y salir

► Seleccione opcion (1-6):

GESTION DE HABITACIONES

- Ver todas las habitaciones
- Ver habitaciones disponibles
- Cambiar estado de habitacion
- Volver al menu principal

► Seleccione opcion (1-4):

ESTADISTICAS

- Reporte de ocupacion
- Ingresos totales
- Ocupacion por tipo de habitacion
- Huesped con mas noches
- Volver al menu principal

► Seleccione opcion (1-5): 1

REPORTE DE OCUPACION

- Total de habitaciones: 13
- Disponibles: 10
- Ocupadas: 2
- En mantenimiento: 1

Porcentaje de ocupacion: 15.4%

GESTION DE HUESPEDES

- Registrar nuevo huesped
- Ver todos los huespedes
- Buscar huesped por DNI
- Eliminar huesped
- Volver al menu principal

► Seleccione opcion (1-5): 1

REGISTRAR NUEVO HUESPED

► DNI (8 digitos) (0=cancelar): 46774979

X Ya existe un huesped con DNI 46774979.

- Nombre: Brisa

► Presione Enter para continuar...

LISTADO DE HABITACIONES

Numero	Tipo	Precio	Estado
101	simple	\$25000	● ocupada
102	simple	\$25000	● disponible
103	simple	\$25000	● disponible
104	simple	\$25000	● disponible
105	simple	\$25000	● mantenimiento
201	doble	\$40000	● disponible
202	doble	\$40000	● disponible
203	doble	\$40000	● disponible
204	doble	\$40000	● disponible
205	doble	\$40000	● disponible
301	suite	\$65000	● disponible
302	suite	\$65000	● disponible
303	suite	\$65000	● ocupada

CHECK-OUT

► DNI del huesped (0=cancelar): 46774979

FACTURA DE ESTADIA

- Huesped: Brisa
- DNI: 46774979
- Habitacion: 303 (suite)
- Noches: 3
- Precio por noche: \$65000

TOTAL A PAGAR: \$195000

► Confirmar check-out? (S/N):

9. Uso de Inteligencia Artificial

En este caso, contemplamos el uso del modelo de inteligencia artificial de Claude Code de Anthropic, el uso principal que se le dio al modelo fue de investigación y planeamiento, proponiendo así un uso más orientado al concepto de cómo podíamos realizar este desarrollo, proporcionando contexto de la materia, conceptos y puntos específicos de la actividad, una vez teniendo esto en nuestro espacio de trabajo, se realizó una versión base del proyecto consultándole de qué manera podíamos realizar el desarrollo.

Una vez que conseguimos dejar funcionando el proyecto, le pedimos que realice una revisión a nivel de código para encontrar posibles fallas a nivel de lógica, detectando así casos que, de lo contrario, hubiesen pasado desapercibidos o generando incongruencia dentro de los datos y/o consultas.

Una vez que teníamos la versión funcionando correctamente, hicimos algunas revisiones manuales y encontrando que podíamos mejorar la visualización de la terminal, que con ayuda de una búsqueda de opciones que pedimos al LLM, se realizaron.

Con esto ya aplicado, el método de comprobación de funcionamiento que ocupamos, se centró en casos de usos, como podíamos reservar, ver clientes o habitaciones habilitadas, opción por opción.

10. Trabajo colaborativo

¿Cómo nos dividimos las tareas?

Integrante	Responsabilidad
Peralta Luciano Ivan	Desarrollo
Ramírez Pablo Nicolas	Video
Rojas Gonzalo Damián	Desarrollo
Vaernet Bianca Estefania	Busqueda de Campo
Zárate Brisa Yael	Busqueda de Campo

11. Conclusión

La realización de este Trabajo Final Integrador nos permitió poner en práctica los conocimientos adquiridos durante la materia, aplicando conceptos como funciones, estructuras de control, validaciones y modularización para desarrollar un sistema funcional en Python. Además, fortalecimos nuestras habilidades de trabajo en equipo, organización del código y uso de herramientas como GitHub e Inteligencia Artificial como apoyo al

desarrollo. En conjunto, esta experiencia nos permitió consolidar los contenidos aprendidos y adquirir una mejor comprensión del proceso de desarrollo de software.

12. Anexos

Link del repositorio

<https://github.com/GonzaloRojas14/Alg-tp-python-d9>

Video de demostración:

https://drive.google.com/drive/folders/1U2O9F8pu8pWSeY4WsnRj5wopncHMW5NG?usp=drive_link